

Camera-Gesture Control of a Bluetooth Smart Car

Embedded Systems, Cyber-Physical Systems and Robotics Project Report

Guanru Chen, Zhixin Huang, Shujun Liao, Zhenkai Lin, Litian Liu, Tonglin Liu,
Yueyang Zhang, and Yiheng Zou

July 2026

Abstract — This project implements a camera-gesture interface for a four-wheel robotic car. A host computer acquires webcam frames, MediaPipe estimates 21 hand landmarks, and a deterministic geometric classifier maps seven gestures to motion commands. The host transmits each command as one ASCII byte over a Bluetooth Classic Serial Port Profile link. An ESP32 receives the byte and drives four DC motors through H-bridge inputs. The implementation includes a keyboard diagnostic controller, suppression of duplicate commands, a stop interval between changes of motion, disconnection checks, and a 0.5-second stop timeout when no valid gesture is visible. Unit tests exercise gesture classification and command sequencing. This report derives the architecture and behavior from the submitted source code. It also identifies limitations: there is no closed-loop vehicle sensing, authentication is inherited from Bluetooth pairing, speed is not controlled by PWM, and physical response time and range have not yet been measured.

1 Introduction

Remote-controlled vehicles are a compact example of a cyber-physical system: a human intention is sensed, converted into a discrete command, transmitted through a communication channel, and finally realized as mechanical motion. Failures in any stage are observable in the physical world. The project objective was therefore not merely to make four motors turn, but to connect perception, communication, embedded control, and actuation into one understandable pipeline.

The implemented prototype replaces buttons or a joystick with hand gestures captured by an ordinary camera. Computation is split between a host computer and an ESP32 on the car. The host performs the expensive vision task and sends a small command; the microcontroller performs time-critical GPIO control. Bluetooth Classic SPP exposes a virtual serial link between these devices [1,2]. The car advertises under the name `roboS`.

The concrete contributions are:

- a seven-command, one-byte control protocol;
- a scale-tolerant geometric gesture classifier built on MediaPipe landmarks;
- safety-oriented host behavior for lost tracking and motion transitions;
- ESP32 firmware that maps commands to four motor directions; and
- automated tests for the classifier and serial command scheduler.

Sections 2–6 present requirements, architecture, gesture recognition, and embedded implementation. Section 7 reports what is verifiable from tests and separates it from physical measurements that remain to be collected. Sections 9 and 10 discuss limitations and future work.

2 Requirements and Scope

The system requirements were reconstructed from the source tree and user-facing command descriptions. Table 1 distinguishes functional behavior from safety and engineering qualities.

The project is deliberately teleoperated rather than autonomous. The camera observes the operator’s hand, not the road. There are no obstacle, wheel-speed, position, or battery sensors in the submitted software. Consequently, the system cannot verify that a requested movement happened, compensate for unequal motors, or stop in response to an obstacle. These are scope boundaries rather than hidden capabilities.

3 System Architecture

Figure 1 shows the end-to-end data and energy paths. Keeping vision on the host avoids deploying a camera model on the microcontroller and allows OpenCV feedback on a normal display. Keeping motor control on the ESP32 reduces the radio message to a single byte and makes actuation independent of host GPIO hardware.

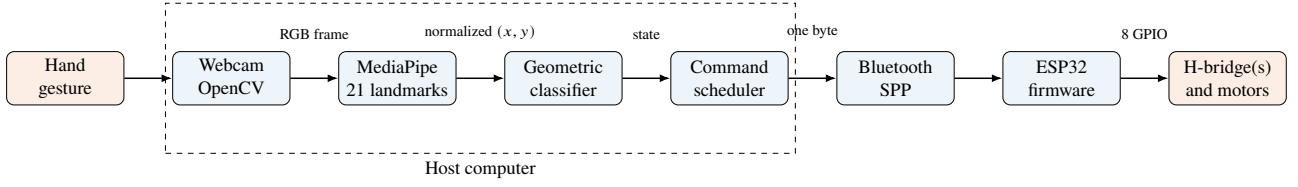


Figure 1 Implemented control pipeline. Vision and command scheduling execute on the host; command decoding and actuation execute on the car.

Table 1 Implemented project requirements.

ID	Requirement
F1	Recognize forward, backward, arc-left, arc-right, spin-left, spin-right, and stop gestures.
F2	Transmit the selected action over Bluetooth serial.
F3	Convert each valid command into deterministic wheel directions.
F4	Provide keyboard control for integration testing without vision.
S1	Stop after 0.5 s without a valid gesture.
S2	Insert a stop before changing between two movement commands.
S3	Stop explicitly on normal user exit and report serial/camera failures.
Q1	Avoid repeated traffic for an unchanged gesture.
Q2	Keep perception independent of car firmware through a small protocol.

Table 2 ESP32 motor-direction pin assignment from `firmware.ino`.

Wheel	Input 1	Input 2
Left front	GPIO 18	GPIO 19
Left rear	GPIO 32	GPIO 33
Right rear	GPIO 14	GPIO 27
Right front	GPIO 26	GPIO 25

3.1 Hardware

The assembled build consists of an ESP32 development board, four DC gear motors and wheels, a car chassis, two L298N dual full-bridge motor-driver modules, a two-cell 18650 battery pack, jumper wires, and a host webcam. The two L298N modules provide the four motor channels required by the eight firmware direction pins (two pins per motor). The L298 is a dual full-bridge driver intended for DC motors and accepts logic-level direction inputs [4]. The exact motor model is not marked on the assembled prototype.

The GPIO mapping is given in Table 2. All pins are configured as outputs and initialized LOW in `setup()`, after which `stopCar()` reinforces the stopped state.

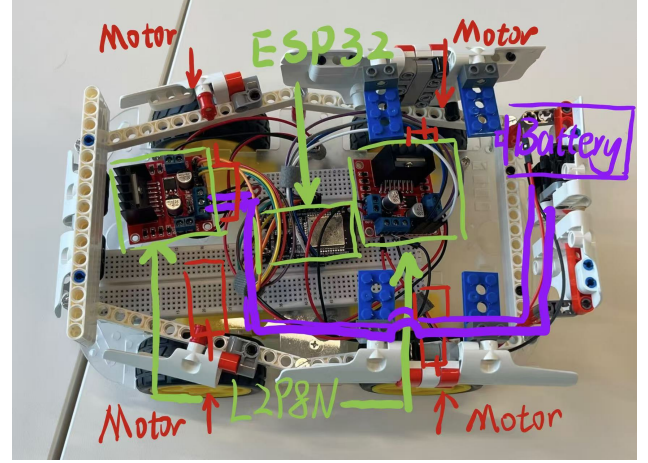


Figure 2 Annotated photograph of the assembled car. It identifies the ESP32, two L298N motor-driver modules, the two-cell battery pack, and the four drive motors.

3.2 Software decomposition

The Python package contains two entry points. `gcsc-controller` maps keyboard keys to protocol bytes and is useful for testing Bluetooth, firmware, and mechanics independently of the camera. `gcsc-gesture-controller` adds webcam capture, hand tracking, classification, preview, and the lost-gesture watchdog. Both reuse `CommandSender`; therefore rate limiting and safe transitions have one implementation.

The firmware is intentionally small. It initializes Bluetooth and GPIO, then polls `ESP_BT.available()`. When a byte is available, a `switch` invokes one of seven motion functions. Unknown bytes cause no state change. This simplicity makes the embedded behavior easy to inspect, although it also means that communication loss is not independently timed out on the car.

4 Gesture Recognition

MediaPipe provides a 21-landmark hand representation, including wrist, metacarpophalangeal (MCP), proximal interphalangeal (PIP), and fingertip indices [3]. The

program tracks at most one hand with detection confidence 0.6 and tracking confidence 0.5. Each webcam frame is mirrored for intuitive interaction, converted from OpenCV's BGR ordering to RGB, and processed by MediaPipe. The preview can display both the image and landmark skeleton.

The submitted classifier does not train an additional machine-learning model. Instead, it applies geometric rules to normalized image coordinates. Let $p_i = (x_i, y_i)$ denote landmark i , and

$$d(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}. \quad (1)$$

A non-thumb finger with PIP landmark p and tip landmark t is considered extended when

$$d(0, t) > 1.18 d(0, p), \quad (2)$$

and folded when

$$d(0, t) \leq 1.08 d(0, p). \quad (3)$$

The gap between 1.08 and 1.18 is a deliberate ambiguity region: a marginal finger is neither confidently folded nor extended. Distances are normalized implicitly because both sides of each comparison scale with the apparent hand size.

4.1 Gesture decision rules

Three hand shapes are relevant. A fist, defined as all four non-thumb fingers folded, maps directly to stop. An open palm, with all four fingers extended, maps to a spin only when its middle finger points predominantly left or right. A pointing hand requires the index finger extended and the middle, ring, and little fingers folded. Its direction is computed from index MCP to index tip. A minimum displacement of 0.08 in normalized image coordinates rejects uncertain short vectors.

For a pointing vector $(\Delta x, \Delta y)$, the dominant axis determines the command. Horizontal positive is right and horizontal negative is left. Because image y increases downwards, vertical positive is backward and vertical negative is forward. For an open palm, horizontal positive/negative selects right/left spin. Vertical open palms are ignored. Table 3 summarizes the mapping.

This rule-based method is transparent and inexpensive. Its limitations are also transparent: it ignores the thumb, uses only 2-D coordinates, assumes one visible hand, and has fixed thresholds rather than user calibration. Foreshortening, partial occlusion, poor lighting, or an oblique hand can change the distance ratios.

Table 3 Recognized gestures and protocol output.

Observed hand pose	Action	Byte
Index up; others folded	Forward	F
Index down; others folded	Backward	B
Index left; others folded	Arc left	L
Index right; others folded	Arc right	R
Open palm pointing left	Spin left	A
Open palm pointing right	Spin right	D
Fist	Stop	S
Invalid/no pose for 0.5 s	Stop	S

Table 4 Application protocol and wheel semantics. F, B, and 0 denote forward, backward, and stopped wheel motion.

Byte	Motion	LF	LR	RF	RR
F	Forward	F	F	F	F
B	Backward	B	B	B	B
A	Spin left	B	B	F	F
D	Spin right	F	F	B	B
L	Arc left	F	B	F	F
R	Arc right	F	F	F	B
S	Stop	0	0	0	0

5 Communication and Safety

The ESP32 uses Bluetooth Classic SPP, which provides serial communication over a virtual link [2]. The host opens the exposed serial device at 115200 baud. Each application-level message is exactly one ASCII byte, so no parsing, delimiter, checksum, or acknowledgement is required. Table 4 defines its semantics.

The arc implementation deserves attention. For a left arc, the left-front motor moves forward but the left-rear motor moves backward while both right motors move forward; the right arc is symmetric. This is the precise submitted behavior, not the more common implementation that slows or stops both wheels on the inside. It should be validated on the assembled chassis because wheel placement and wiring determine the resulting trajectory.

5.1 Command scheduling

`CommandSender` stores the most recently transmitted byte and transmission time. Duplicate states are suppressed. A minimum command interval of 0.2 s limits rapid writes. More importantly, when one movement command changes to another, the scheduler waits until the interval permits transmission, sends S, waits 0.3 s, and then sends the new movement byte. Thus a forward-to-backward transition becomes F, S, B, reducing abrupt electrical and mechanical reversal.

Table 5 Direction truth table implemented for every motor.

Direction	IN1	IN2
Forward	HIGH	LOW
Backward	LOW	HIGH
Stop	LOW	LOW

Explicit stop commands are forced: they are sent even when the previous state was also stop, after respecting the minimum interval. The gesture application sends stop when no valid gesture has been observed for 0.5 s. It also checks the serial descriptor for disconnect/error conditions each loop. On `q`, Escape, or a normal keyboard-controller exit, it sends a final stop.

These mechanisms improve host-side safety, but they are not a complete fail-safe. If the host process crashes, the Bluetooth connection disappears, or packets stop after a movement byte, the firmware retains the previous motor outputs indefinitely. A firmware watchdog is therefore the highest-priority improvement.

6 Embedded Motor Control

Each motor is driven by two digital inputs. The helper `setMotor(in1,in2,dir)` implements the truth table in Table 5. Higher-level functions group the four wheels and avoid duplicating GPIO operations. The abstraction hierarchy is: individual H-bridge input pair, named wheel, four wheels, paired sides, and vehicle maneuver.

No PWM calls or speed commands exist in the firmware. Motors therefore receive binary direction control, with actual speed determined by battery voltage, driver losses, motor load, and motor characteristics. Speed characters such as digits are not part of the implemented protocol. Adding speed control would require enable-pin PWM (or PWM on suitable bridge inputs), a protocol extension, and testing of low-duty-cycle starting behavior.

The firmware prints received bytes to its 115200-baud USB serial console. This is useful during bench testing: an operator can verify that Bluetooth reception occurred even when the vehicle is lifted so wheels cannot propel it. Unknown characters are ignored, and boot initializes all outputs low.

7 Development and Verification

The Python project targets Python 3.12 and declares MediaPipe 0.10.21, OpenCV 4.11 or newer, and py-

Table 6 Verification evidence available in the source repository.

Test area	Evidence
Pointing classification	Four directions
Open-palm classification	Two spins; vertical rejected
Stop classification	Fist maps to S
Input validation	Short landmark list rejected
State handling	Duplicate command skipped
Motion transitions	Intermediate stop asserted
Serial write	Byte payload and short-write error

Serial 3.5 or newer. Two command-line programs are installed from the same package. The firmware can be compiled for `esp32:esp32:esp32` with Arduino CLI. Separating a keyboard controller from the gesture controller enables staged integration:

1. lift the wheels and verify firmware startup and pin states;
2. send keyboard commands and check serial debug output and wheel direction;
3. verify stop and reversal behavior;
4. run camera control with preview; and
5. conduct floor tests at low risk in an open area.

7.1 Automated tests

The repository contains unit tests for host logic. Gesture fixtures synthesize 21 normalized points for pointing hands, open palms, and fists. Tests cover all seven gestures, reject vertical open palms, reject incomplete landmarks, and reject a pointing pose when the other fingers are extended. Scheduler tests verify immediate first transmission, duplicate suppression, forced stops, minimum spacing, and `movement--stop--movement` sequencing. Serial tests verify single-byte writes and error reporting for incomplete writes.

These are deterministic software tests, not measurements of physical performance. They demonstrate that specified inputs produce expected outputs in isolation, but do not quantify vision accuracy on real users, Bluetooth range, end-to-end delay, steering symmetry, current draw, or stopping distance.

8 Challenges and Design Trade-offs

Vision ambiguity. A gesture interface must distinguish intentional commands from transitional poses. The classifier addresses this with finger-ratio thresholds, a dead zone, and an invalid-pose timeout. The trade-off is that an invalid pose can leave the previous command active for up to 0.5 s.

Responsiveness versus motor stress. Immediate direction changes feel responsive but can cause current spikes and drivetrain shock. The 0.3 s stopped interval is conservative and deterministic, although it adds latency whenever a user changes motion.

Simple protocol versus observability. One-byte commands minimize implementation complexity. Without sequence numbers, acknowledgement, or telemetry, however, the host cannot know whether the ESP32 received or executed a command.

Open-loop actuation. Manufacturing variation, load, and battery state cause unequal wheel speeds. Binary GPIO control cannot compensate. PWM and wheel encoders would permit calibration and closed-loop velocity control.

Mechanical integration. Jumper wires and battery connections are exposed to vibration and motor electrical noise. Secure connectors, strain relief, decoupling, and a fixed enclosure are important even though they are not represented in software.

9 Limitations and Future Work

The following improvements are prioritized by safety and architectural value:

1. **Firmware communication watchdog.** Record the last valid command time and stop the motors if no fresh command arrives within a short deadline. The host should then periodically refresh active commands instead of suppressing them forever.
2. **Validated motor-driver wiring.** Document a complete schematic, driver count, shared ground, battery voltage, enable pins, and protection/decoupling components.
3. **Variable speed and calibration.** Add PWM and per-side calibration to reduce drift. Extend the protocol in a backward-compatible way, for example with framed command and speed fields.
4. **Feedback sensors.** Wheel encoders enable closed-loop speed; an ultrasonic or time-of-flight sensor

can enforce local obstacle stopping even if the host fails.

5. **Gesture robustness.** Gather user data, tune thresholds, add temporal voting or hysteresis, and consider handedness and depth. A configurable neutral gesture could require deliberate arming before movement.
6. **Communication security.** Define pairing and authorization procedures and reject control from unintended hosts. Bluetooth pairing alone should not be treated as a complete application safety policy.

The desktop application can also be made portable by improving serial-port discovery. The current default `/dev/cu.robos` is appropriate to one macOS pairing configuration but not a universal Windows or Linux device name; `--port` and the `GCSC_PORT` environment variable are the present escape hatches.

10 Conclusion

The project demonstrates an end-to-end cyber-physical control chain from camera image to vehicle motion. MediaPipe supplies normalized hand landmarks, a compact geometric classifier converts them into seven states, Bluetooth SPP carries a one-byte command, and the ESP32 translates that command into four motor directions. The code is modular enough to test perception, transport, and actuation separately, and its host-side scheduler provides meaningful safeguards against rapid reversals and lost gestures.

At the same time, source-level analysis exposes the boundary between a functioning prototype and a dependable robot. Control remains open loop; speed is binary; physical performance has not been quantified; and the embedded controller lacks an independent communication timeout. The strongest next iteration is therefore not a larger gesture vocabulary, but a firmware watchdog, documented wiring, and measured system behavior. Those additions would turn the current demonstrator into a more robust and reproducible embedded-systems project.

Reproducibility Notes

The source of truth for this report is the submitted repository: `firmware/firmware.ino`, `tools/controller.py`, `tools/gesture_controller.py`, and the two unit-test modules. The gesture program

is run after pairing with roboS using `uv run gcsc-gesture-controller --port <device>`. The keyboard diagnostic uses `uv run gcsc-controller --port <device>`. Firmware build and upload instructions are included in the repository's firmware README.

References

- [1] Espressif Systems. Bluetooth: Arduino esp32 documentation.
- [2] Espressif Systems. Bluetooth spp api: Esp-idf programming guide.
- [3] Google. Mediapipe hand landmark enumeration.
- [4] STMicroelectronics. *L298 Dual Full-Bridge Driver Datasheet*, 2023.